강화학습 Day3 HW1 레포트

로빛 예비단원 주호진

1) 과제

구분	과제 요건
구현	Ubuntu CPU, 4입력·2출력 Q-Network, Replay·Target 구현 ε-greedy, 종료·시간 제한 구분, 모델 저장·재로딩
평가·분석	별도 50회 greedy 평가, 평균·표준편차·최소·최대 보고 무작위 정책 비교, 설정 1개 변경 실험 및 원인 분석
시각화·제출	학습 보상·평가 보상·TD loss·ε 그래프, 실행 영상 MP4 GitHub 코드, 모델, 보고서, README, requirements.txt
설명·준수	TD target 수치 계산, Replay·Target 역할 설명 Codex·GPT 사용 부분과 사유 명시, 본인 코드 설명 필수

2) 코드

Ppt 에 있는 코드를 기반으로 실험하였고, 시각화 그래프는 Gemini 를 사용했다.

```
steps = 40000
decay_step = 15000
epsilon_min = 0.1
discount_factor = 0.99
warmup = 1000
online_step = 4
target_step = 1000
memory_size = 20000

lr = 0.0005
batch_size = 64

seed = 0
```

수정할 파라미터들을 위로 빼주었다.

```
def make_net():
    return nn.Sequential(
    nn.Linear(4, 32), nn.ReLU(),
    nn.Linear(32, 32), nn.ReLU(),
    nn.Linear(32, 32), nn.ReLU(),
    nn.Linear(32, 2)
    )
```

초반 ppt 에 있던 신경망 사용 -> 64 에서 32 로 축소하고 레이어를 더 쌓음

```
ns, r, term, trunc, _ = env.step(action)

cart_pos = ns[0]
cart_vel = ns[1]
pole_angle = ns[2]
pole_angular_vel = ns[3]

r = 1.0

if abs(ns[2]) > 0.04:
    r += -1
if abs(ns[0]) > 1.6:
    r += -1

if term:
    r += -20.0
elif trunc:
    r += 100.0
```

Ns[0] ~ ns[3] 값을 보면서 실패하는 시점에서 나타나는 값의 공통적인 부분을 조건으로 걸어서 reward 를 깎았다. 실패 시 reward 감소, 성공 시 reward 추가를 하여 성공하게끔 보상을 조정했다.

```
print(f"step: {step} | episode: {episode_count}, reward: {current_e
print(f"{ns[0]:.2f}, {ns[1]:.2f}, {ns[2]:.2f}, {ns[3]:.2f}")
```

실패하는 경우

1. 각도가 틀어짐 -> 각도에 대한 벌

2. 각도를 유지하기 위해 한쪽 방향으로 가다가 맵 양 사이드로 탈출 -> 양 사이드 쪽에 갔을 때 벌

카트 속도와 각속도 값에 따라 reward 를 조정했을 때 학습이 더 안되는 것 같아 제외시켰다. 막대가 살짝 기울어졌을 때 보정하기 위해 속도가 바뀔 때와, 실패하는 경우가 구분이 잘 안되는 부분이 학습에 방해가 되는 것 같았다.

3) 결과

초기값 -> reward 200 이상으로 올라가지 않음

term 시 벌, trunc 시 상 -> 200 이상으로 올라가나 여전히 정체

ns[0] ~ ns[3]값을 reward 에 반영

i) 위치와 각도 반영 -> 약간 개선

끝으로 가서 죽는 경우가 많아짐

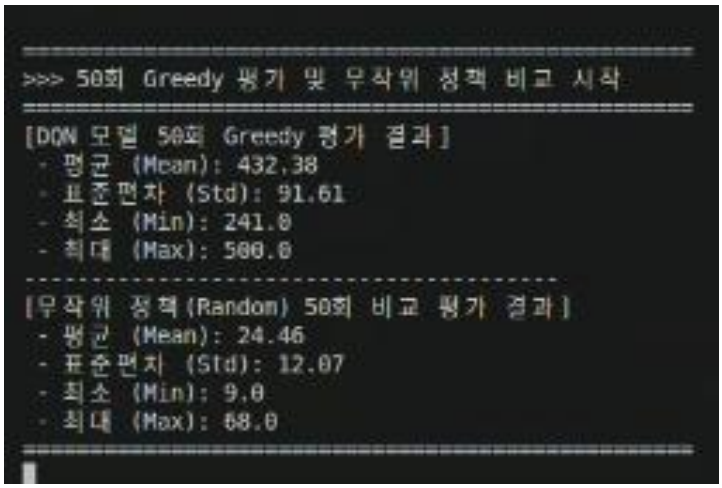
ii) 위치에 대한 벌 강화 -> 막대가 쓰러져서 죽음

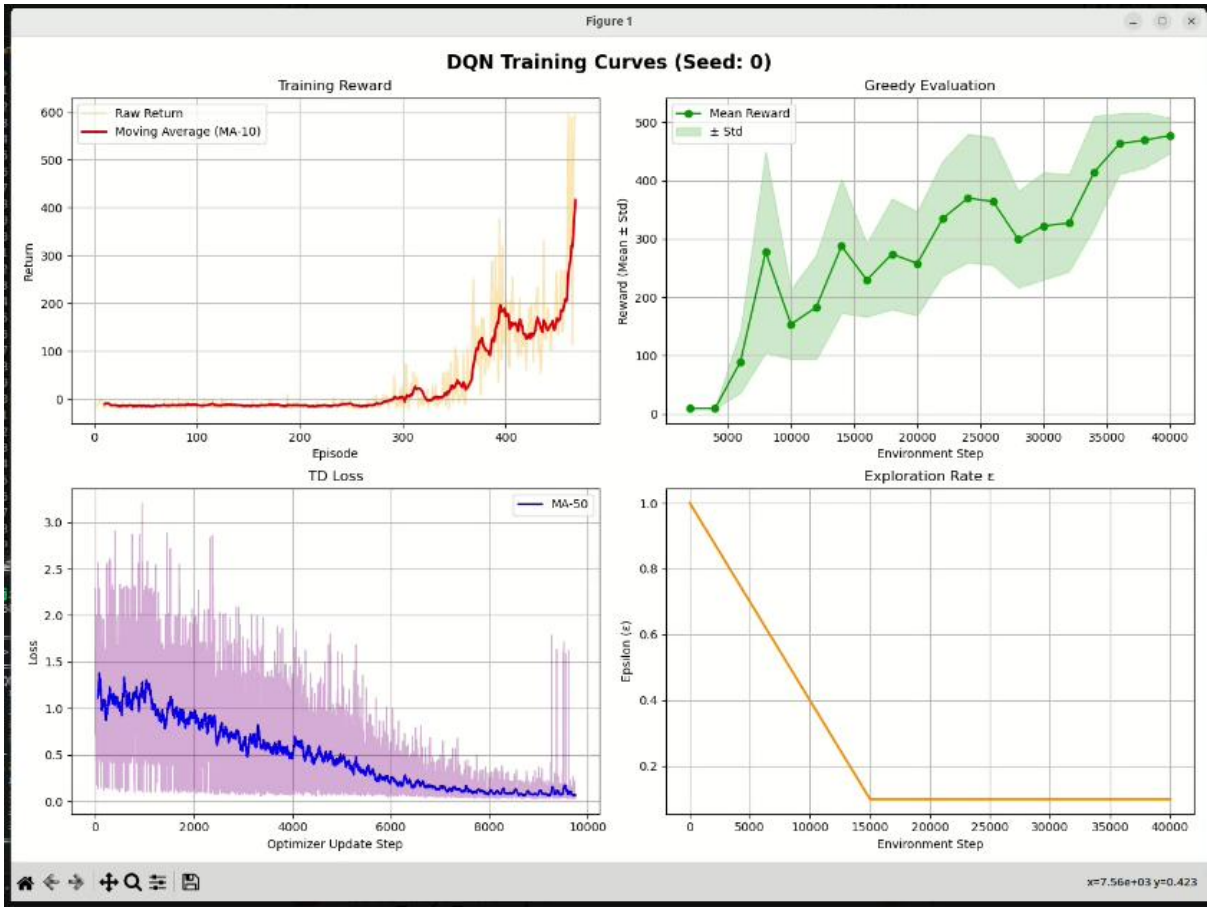
iii) 막대 각도에 대한 벌 강화 -> 둘 중 reward 감소가 작은쪽으로 편향

iv) 죽기 직전 값을 가졌을 때 벌 + reward 가중치 동일 -> 가장 잘 됨

학습률 0.0001

https://drive.google.com/file/d/1PCqtYn-D0f4RLRdT9uCtuWmYl2CLOHh1/view?usp=sharing

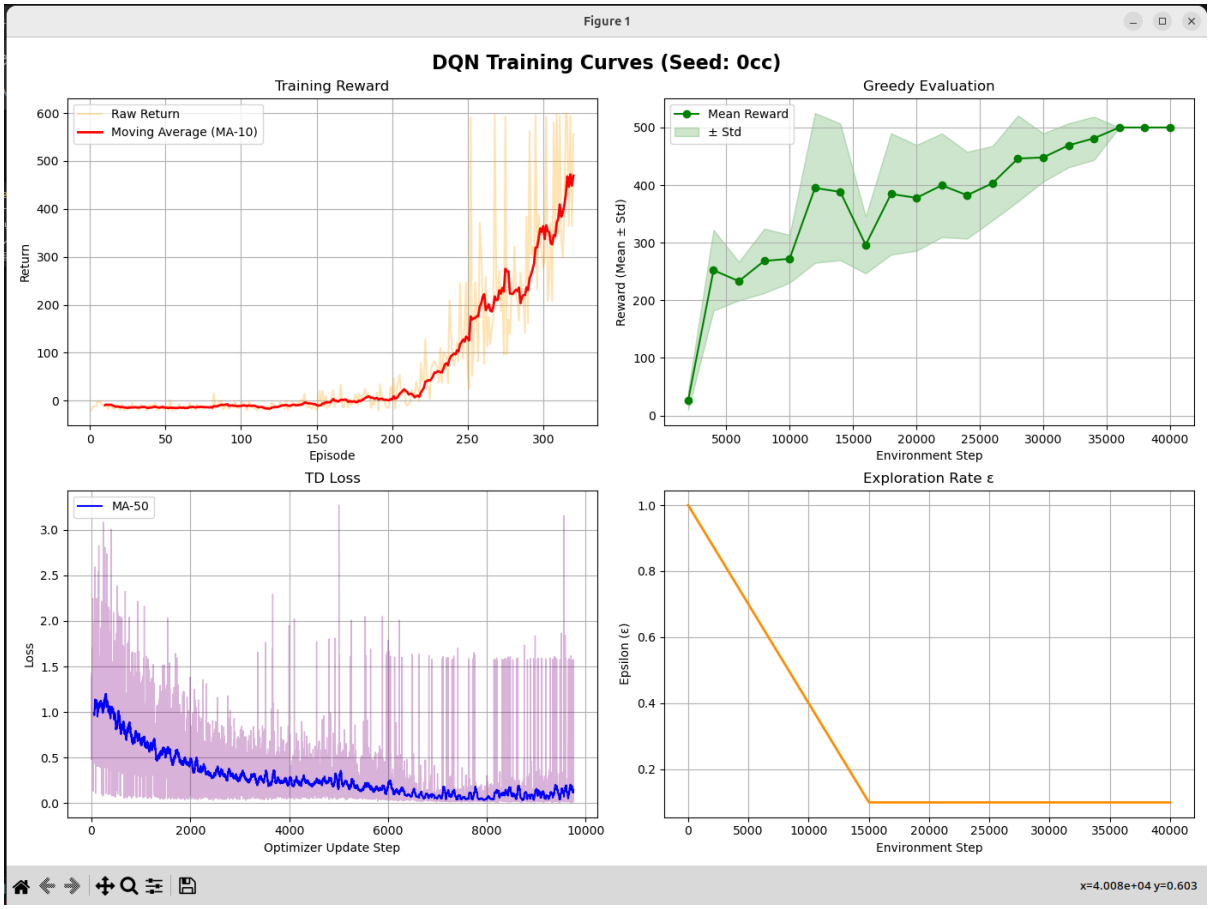




학습률 0.0005

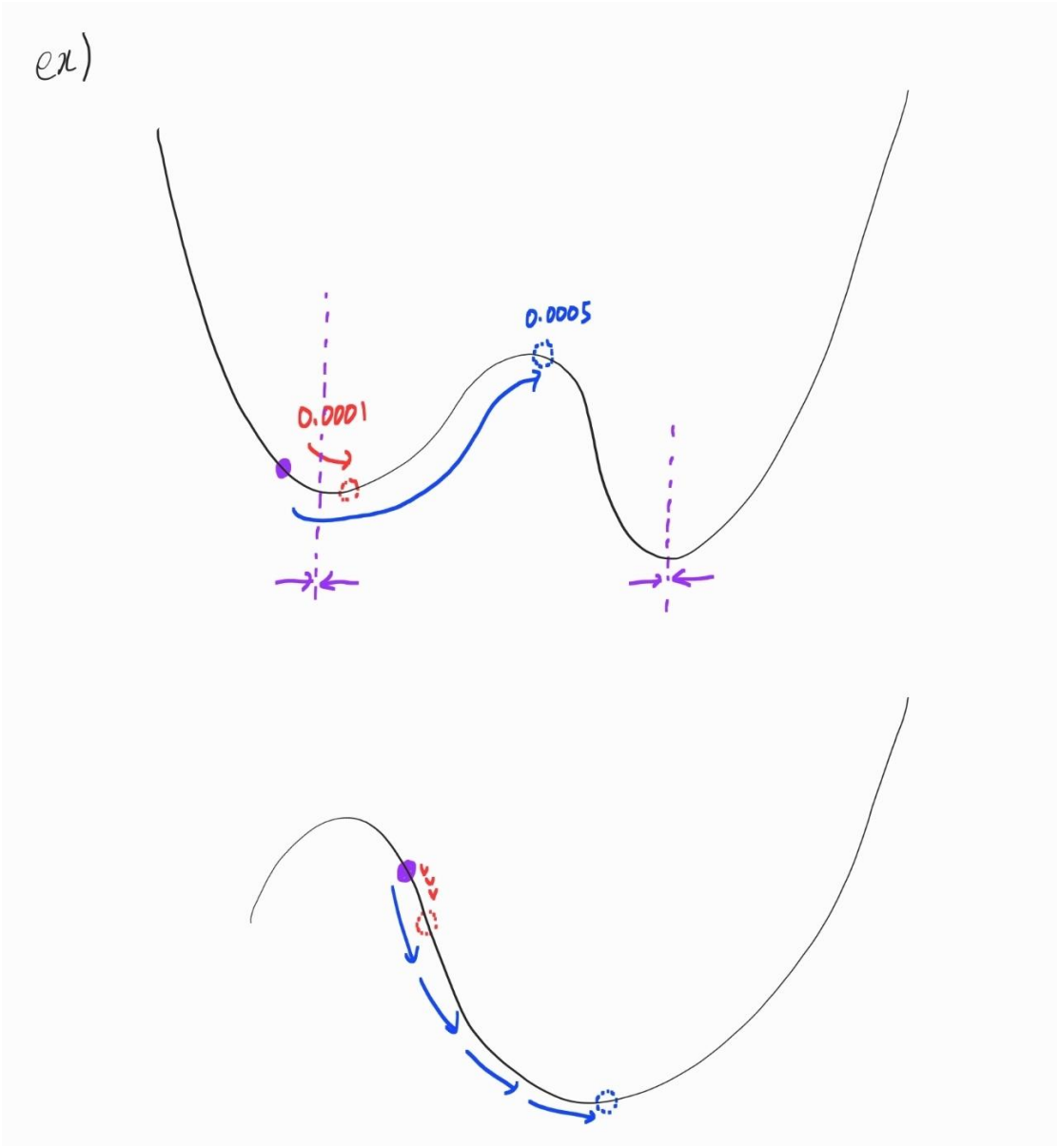
https://drive.google.com/file/d/1XU053ueanG8D3jIoFrztzCWuhz2Q1QPP/view?usp=sharing

```
============================================================
>>> 50회 Greedy 평가 및 무작위 정책 비교 시작
============================================================
[DQN 모델 50회 Greedy 평가 결과]
 - 평균 (Mean): 500.00
 - 표준편차 (Std): 0.00
 - 최소 (Min): 500.0
 - 최대 (Max): 500.0
----------------------------------------
[무작위 정책(Random) 50회 비교 평가 결과]
 - 평균 (Mean): 22.40
 - 표준편차 (Std): 12.01
 - 최소 (Min): 9.0
 - 최대 (Max): 65.0
============================================================
```



학습률 0.0001 -> 0.0005 일 때 더 잘되는 결과가 나왔다.

기존에 테스트 했을 때는 학습률이 더 낮을 때 좋은 결과가 나왔었는데, 보상을 조정한 뒤 학습률을 올렸을 때 기존 local minimum 을 탈출하고 더 최적화된 지점으로 수렴하여 더 좋은 결과값이 뜬 것이라고 예측할 수 있다. 혹은 같은 local minimum 을 향해 수렴하는데, 학습률이 낮은 경우 극솟점으로 수렴해가는 과정 중 step 이 끝났을 수도 있다고 생각한다.



4) 개념 정리

$$TD\ target = R + \gamma(1-d)\max Q\theta^{-}(s', a')$$

d=1, terminated (맵 탈출, pole 쓰러짐...)

R = 바로 다음 step의 Reward.

γ = discount factor (할인율): 먼 미래일수록 더 작은 값을 반영하도록 함.

1-d = term시 미래 가치가 0이므로 미적용

$\max Q\theta(s', a')$ = 다음 step에서 expect return이 가장 큰 action에 대한 return값

if) r=1, γ=0.9, d=0, Q=100

TD target = 1 + 0.9×1×100 = 1+90 = 91

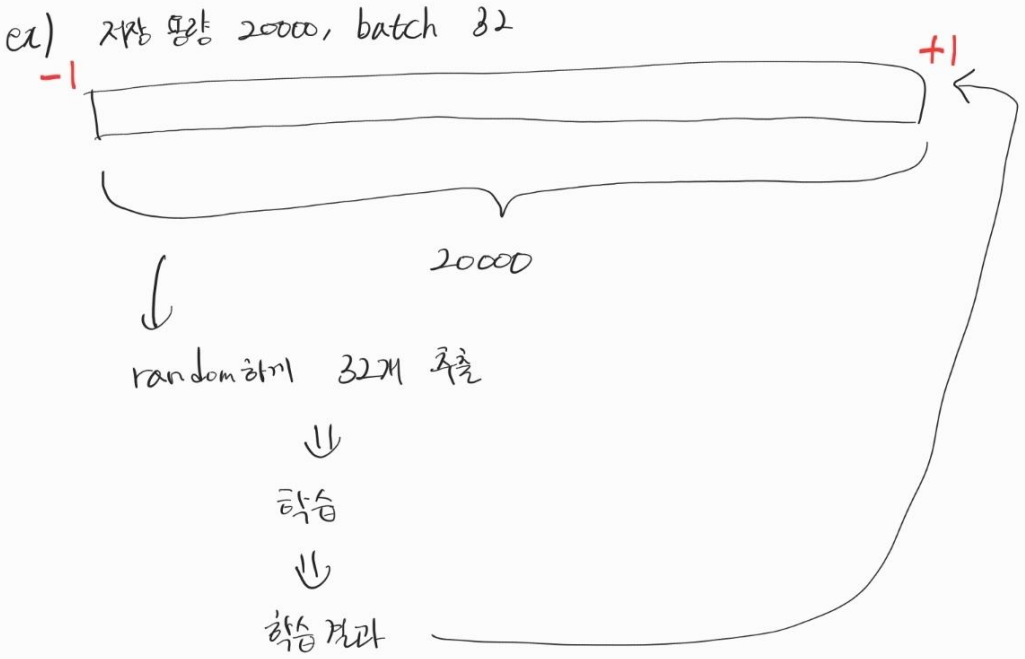
if) 위 조건에서 d=1

TD target = 1 + 0.9×0×··· = 1

Replay Buffer

저장 단위: $(S, a, r, S', \text{terminated})$

저장된 값에서 batch size 만큼의 무작위 샘플을 뽑아 학습에 활용



최근값이 아닌 랜덤한 값을 뽑아쓰면서 편향이 줄어듦

Target Network

Q값을 업데이트할 때 쓰는 Target이 Q값에 영향을 받음.

$$Target = R + \gamma(1-d)\max Q\theta^{-}(s', a')$$

ex) Target Network 미적용, $d=0$, $\gamma=0.9$

예측 Q	Target	Loss
1	$1+0.9\times1=1.9$	0.9
1.9	$1+0.9\times1.9\simeq2.7$	0.8
2.7	$1+0.9\times2.7\simeq3.4$	0.7
⋮	⋮	

점점 커짐 (발산)

따라서 매번 Target을 업데이트하지 않고 일정 주기마다 업데이트하여 위 같은 경우를 최소화함.